

PRAKTIKUM MACHINE LEARNING

# Bab 6

## KNN dan Naive Bayes

Kumpulan Praktikum Machine Learning

|             |                                  |
|-------------|----------------------------------|
| Nama        | <b>Ahmad Dandi Subhani</b>       |
| NPM         | <b>202343500126</b>              |
| Kelas       | <b>R7B</b>                       |
| Mata Kuliah | <b>Machine Learning</b>          |
| Dosen       | <b>Nurfidah Dwitianti, M.Si.</b> |

2 Oktober 2026

## Daftar Isi

---

|                                                                   |          |
|-------------------------------------------------------------------|----------|
| Sel 0 - Persiapan Awal (Khusus Google Colab) .....                | 2        |
| <b>KNN dan Naive Bayes .....</b>                                  | <b>3</b> |
| <b>Ringkasan Konsep .....</b>                                     | <b>4</b> |
| <b>6.17 Latihan Praktikum (Modul Bab 6) .....</b>                 | <b>4</b> |
| Praktikum 1: KNN vs Naive Bayes .....                             | 4        |
| Praktikum 2: Klasifikasi Teks dengan Multinomial Naive Bayes..... | 7        |
| <b>Kesimpulan Bab 6 .....</b>                                     | <b>9</b> |

## Tools yang Digunakan

---

**Python** 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

### Sel 0 - Persiapan Awal (Khusus Google Colab)

Kalau notebook ini dibuka di Google Colab, jalankan sel di bawah ini dulu (atau langsung **Runtime > Run all**) untuk memastikan semua paket yang dipakai bab ini tersedia. Kalau dijalankan di Jupyter lokal dan paketnya sudah terpasang, sel ini tidak mengubah apa-apa.

In [1] - Kode Program

```
1 # Sel 0 - Persiapan awal: cek paket yang dipakai bab ini.
2 # scikit-learn, pandas, numpy, dan matplotlib sudah tersedia di environment,
3 # jadi sel ini hanya memastikan import-nya berjalan tanpa error.
4 import importlib
5
6 for _paket in ["sklearn", "pandas", "numpy", "matplotlib"]:
7     importlib.import_module(_paket)
8     print(f"OK: {_paket} tersedia.")
9
10 print("Paket siap.")
```

OK: sklearn tersedia.  
OK: pandas tersedia.  
OK: numpy tersedia.

OK: matplotlib tersedia.  
Paket siap.

# Praktikum Machine Learning: Bab 6

## KNN dan Naive Bayes

|                    |                           |
|--------------------|---------------------------|
| <b>Nama</b>        | Ahmad Dandi Subhani       |
| <b>NPM</b>         | 202343500126              |
| <b>Kelas</b>       | R7B                       |
| <b>Mata Kuliah</b> | Machine Learning          |
| <b>Dosen</b>       | Nurfidah Dwitianti, M.Si. |

**Catatan:** notebook ini saya jalankan di Google Colab / Jupyter Notebook. Bab 6 ini mengerjakan **Latihan Praktikum Modul Bab 6 bagian 6.17**, memakai dataset **Breast Cancer** dari `sklearn.datasets.load_breast_cancer` dan data teks kecil buatan sendiri. Tidak perlu file CSV apa pun.

## Ringkasan Konsep

---

- **KNN (K-Nearest Neighbors)** adalah algoritma lazy learning: tidak ada fase pelatihan eksplisit, model hanya menyimpan data latih. Saat prediksi, ia mencari **k** tetangga terdekat (biasanya dengan jarak Euclidean) lalu mengambil suara terbanyak. Nilai **k** yang kecil bikin model sensitif ke noise (varians tinggi), nilai **k** yang besar bikin batas keputusannya terlalu mulus (bias tinggi).
- **Scaling itu wajib untuk KNN.** Karena KNN bekerja dengan jarak, fitur yang skalanya besar (misal 1000-an) akan mendominasi fitur yang skalanya kecil (misal 0-1). Solusinya: standarisasi dulu dengan `StandardScaler` sebelum menghitung jarak.
- **Naive Bayes** adalah classifier probabilistik berbasis **teorema Bayes**: ia menghitung peluang tiap kelas berdasarkan fitur yang diamati. Disebut "naive" karena mengasumsikan semua fitur **saling independen** (tidak saling memengaruhi), asumsi yang jarang benar di dunia nyata tapi ternyata sering tetap menghasilkan model yang bagus.
- **GaussianNB vs MultinomialNB.** `GaussianNB` dipakai untuk fitur numerik kontinu dengan asumsi tiap fitur berdistribusi normal (Gaussian) di tiap kelas. `MultinomialNB` dipakai untuk data hitungan/frekuensi, paling umum untuk klasifikasi teks berbasis bag-of-words (misal hasil `CountVectorizer`), dengan parameter `alpha` sebagai Laplace smoothing.
- **Kapan pakai yang mana.** KNN cocok untuk dataset kecil dengan batas keputusan yang tidak linear, tapi prediksinya lambat karena tiap prediksi harus menghitung jarak ke semua data latih. Naive Bayes sangat cepat dan ringan, bagus sebagai baseline dan juara untuk klasifikasi teks.

## 6.17 Latihan Praktikum (Modul Bab 6)

---

Bagian ini mengerjakan dua latihan praktikum dari modul: membandingkan KNN dan Naive Bayes pada dataset yang sama, lalu menerapkan Multinomial Naive Bayes untuk klasifikasi teks.

### Praktikum 1: KNN vs Naive Bayes

**Tujuan:** Mahasiswa mampu membandingkan performa KNN dan Naive Bayes pada dataset yang sama.

**Instruksi:** Gunakan dataset `load_breast_cancer` dari `scikit-learn`. Latih model KNN (dengan **scaling**) dan `Gaussian Naive Bayes`. Bandingkan **akurasi**, **waktu pelatihan**, dan **waktu prediksi** keduanya menggunakan modul `time` Python.

```

1 import time
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6 from sklearn.datasets import load_breast_cancer
7 from sklearn.metrics import accuracy_score
8 from sklearn.model_selection import train_test_split
9 from sklearn.naive_bayes import GaussianNB
10 from sklearn.neighbors import KNeighborsClassifier
11 from sklearn.pipeline import Pipeline
12 from sklearn.preprocessing import StandardScaler
13
14 # 1. Muat dataset Breast Cancer Wisconsin: 569 baris, 30 fitur numerik
15 data = load_breast_cancer()
16 X, y = data.data, data.target
17 print(f"Dataset: Breast Cancer Wisconsin ({X.shape[0]} baris, {X.shape[1]}
18 fitur)")
19
20 # 2. Bagi data: 80% latih, 20% uji
21 X_train, X_test, y_train, y_test = train_test_split(
22     X, y, test_size=0.2, random_state=42
23 )
24 print(f"Data latih: {X_train.shape[0]} baris, data uji: {X_test.shape[0]} baris")
25
26 # 3. Siapkan dua model. KNN dibungkus Pipeline agar scaling ikut di dalamnya.
27 model_knn = Pipeline([
28     ("scaler", StandardScaler()),
29     ("knn", KNeighborsClassifier()),
30 ])
31 model_gnb = GaussianNB()
32
33 # 4. Latih tiap model sambil mengukur waktu latih dan waktu prediksi
34 hasil = []
35 for nama, model in [("KNN (dengan scaling)", model_knn),
36                     ("Gaussian Naive Bayes", model_gnb)]:
37     t0 = time.perf_counter()
38     model.fit(X_train, y_train)
39     waktu_latih = time.perf_counter() - t0
40
41     t1 = time.perf_counter()
42     prediksi = model.predict(X_test)
43     waktu_prediksi = time.perf_counter() - t1
44
45     akurasi = accuracy_score(y_test, prediksi)
46     hasil.append((nama, akurasi, waktu_latih, waktu_prediksi))
47     print(f"{nama}: akurasi={akurasi:.4f}, "
48           f"waktu latih={waktu_latih:.4f} detik, "
49           f"waktu prediksi={waktu_prediksi:.6f} detik")
50
51 # 5. Tabel perbandingan
52 tabel = pd.DataFrame(hasil, columns=["Model", "Akurasi",
53                                     "Waktu latih (detik)",
54                                     "Waktu prediksi (detik)"])

```

```

54 print("\nTabel perbandingan:")
55 print(tabel.to_string(index=False))
56
57 # 6. Grafik batang perbandingan akurasi
58 plt.figure(figsize=(7, 4))
59 plt.bar([h[0] for h in hasil], [h[1] for h in hasil])
60 plt.title("Perbandingan Akurasi: KNN vs Gaussian Naive Bayes")
61 plt.ylabel("Akurasi (data uji)")
62 plt.ylim(0.9, 1.0)
63 for i, h in enumerate(hasil):
64     plt.text(i, h[1] + 0.002, f"{h[1]:.4f}", ha="center")
65 plt.tight_layout()
66 plt.show()

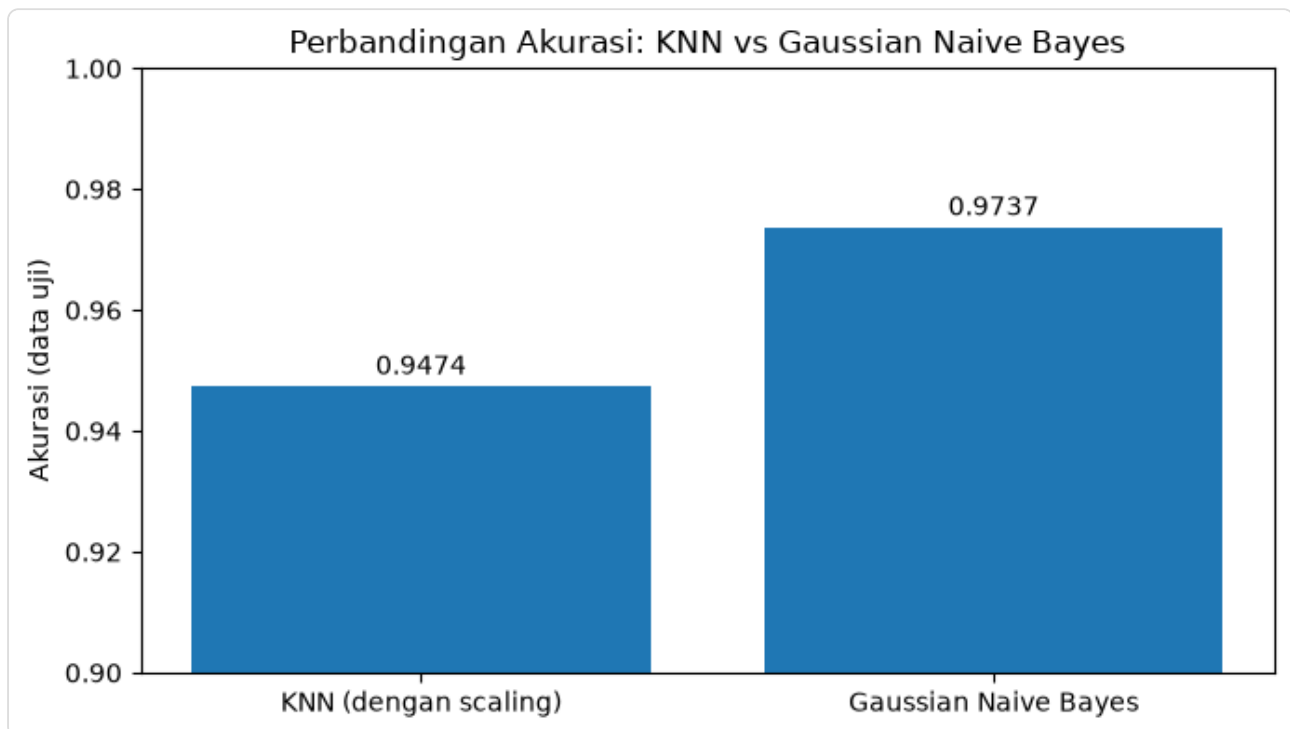
```

Dataset: Breast Cancer Wisconsin (569 baris, 30 fitur)  
 Data latih: 455 baris, data uji: 114 baris

KNN (dengan scaling): akurasi=0.9474, waktu latih=0.0459 detik, waktu prediksi=0.991085 detik  
 Gaussian Naive Bayes: akurasi=0.9737, waktu latih=0.0393 detik, waktu prediksi=0.000610 detik

Tabel perbandingan:

|  | Model                | Akurasi  | Waktu latih (detik) | Waktu prediksi (detik) |
|--|----------------------|----------|---------------------|------------------------|
|  | KNN (dengan scaling) | 0.947368 | 0.045909            | 0.991085               |
|  | Gaussian Naive Bayes | 0.973684 | 0.039258            | 0.000610               |



Gambar 1

### Penjelasan outputnya:

- Dataset Breast Cancer yang dipakai punya 569 baris dan 30 fitur, lalu saya bagi jadi 455 baris data latih dan 114 baris data uji (`test_size=0.2`, `random_state=42`).
- Akurasi di data uji: **KNN (dengan scaling) 0.9474** dan **Gaussian Naive Bayes 0.9737**. Di dataset ini Naive Bayes sedikit lebih akurat daripada KNN.
- Waktu pelatihan keduanya sangat cepat (orde milidetik). GaussianNB sedikit lebih cepat karena saat `fit()` ia hanya menghitung rata-rata dan varians tiap fitur per kelas, sedangkan KNN pada dasarnya hanya menyimpan data latih.
- Perbedaan paling terasa di **waktu prediksi**: KNN butuh sekitar 0.99 detik untuk 114 data uji, sedangkan GaussianNB hanya sekitar 0.0006 detik (lebih dari seribu kali lebih cepat). Wajar, karena KNN itu lazy: tiap prediksi harus menghitung jarak ke seluruh 455 data latih, sedangkan Naive Bayes tinggal menghitung peluang pakai rumus yang sudah diringkas saat training.

## Praktikum 2: Klasifikasi Teks dengan Multinomial Naive Bayes

**Tujuan:** Mahasiswa mampu menerapkan Multinomial Naive Bayes pada data teks sederhana.

Kode di bawah ini mengikuti **Praktikum 6.7** dari modul. Setelah itu saya coba beberapa teks baru tambahan untuk membuktikan modelnya benar-benar bekerja, bukan sekadar menghafal.

### In [3] - Kode Program

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 from sklearn.naive_bayes import MultinomialNB
3
4 teks = ["dapatkan diskon gratis sekarang",
5         "rapat tim jam 10 pagi",
6         "menangkan hadiah gratis klik disini",
7         "laporan keuangan bulan ini", "diskon spesial hari ini",
8         "jadwal kuliah minggu depan"]
9 label = [1, 0, 1, 0, 1, 0] # 1 = spam, 0 = bukan spam
10
11 vectorizer = CountVectorizer()
12 X_teks = vectorizer.fit_transform(teks)
13 model = MultinomialNB(alpha=1.0) # alpha = parameter Laplace Smoothing
14 model.fit(X_teks, label)
15
16 teks_baru = ["diskon gratis hari ini"]
17 prediksi = model.predict(vectorizer.transform(teks_baru))
18 print("Prediksi:", "Spam" if prediksi[0] == 1 else "Bukan Spam")
19
20 # Uji tambahan: dua teks yang jelas spam dan dua yang jelas bukan spam
21 teks_uji = [
22     "selamat anda menang undian berhadiah klik link ini", # spam
23     "dapatkan bonus saldo gratis sekarang juga",           # spam
24     "jadwal rapat dosen jam 9 pagi",                       # bukan spam
25     "laporan praktikum sudah dikumpulkan",                 # bukan spam
26 ]
27 print("\nUji teks tambahan:")
28 for t in teks_uji:
29     p = model.predict(vectorizer.transform([t]))
30     print(f"- {t!r} -> {'Spam' if p[0] == 1 else 'Bukan Spam'}")
```

Prediksi: Spam

Uji teks tambahan:

- 'selamat anda menang undian berhadiah klik link ini' -> Spam
- 'dapatkan bonus saldo gratis sekarang juga' -> Spam
- 'jadwal rapat dosen jam 9 pagi' -> Bukan Spam
- 'laporan praktikum sudah dikumpulkan' -> Bukan Spam

### Penjelasan outputnya:

- Teks dari modul, "diskon gratis hari ini", diprediksi sebagai **Spam**. Masuk akal karena kata "diskon" dan "gratis" paling sering muncul di contoh spam pada data latih.
- Dua teks spam tambahan saya ("selamat anda menang undian berhadiah klik link ini" dan "dapatkan bonus saldo gratis sekarang juga") juga diprediksi **Spam** dengan benar, karena mengandung kata-kata pola spam seperti "menang", "hadiah", "klik", "gratis", dan "diskon".

- Dua teks non-spam ("jadwal rapat dosen jam 9 pagi" dan "laporan praktikum sudah dikumpulkan") diprediksi **Bukan Spam** dengan benar, karena kosakatanya mirip contoh bukan spam di data latih ("rapat", "jadwal", "laporan").
- Parameter  $\alpha=1.0$  (Laplace smoothing) membuat kata yang belum pernah muncul di data latih tetap punya peluang kecil, bukan nol. Jadi model tidak langsung kaku kalau ketemu kata baru.

## Kesimpulan Bab 6

---

- Pada dataset Breast Cancer, **Gaussian Naive Bayes (akurasi 0.9737) sedikit mengungguli KNN dengan scaling (0.9474)**, dan jauh lebih cepat saat prediksi (sekitar 0.0006 detik vs 0.99 detik untuk 114 data uji).
- **Scaling terbukti penting untuk KNN** karena algoritmanya berbasis jarak Euclidean: tanpa `StandardScaler`, fitur dengan angka besar akan mendominasi perhitungan jarak dan merusak hasil.
- **Naive Bayes sangat ringan dan cepat** (waktu latih dan prediksi sama-sama orde milidetik), sehingga cocok dipakai sebagai baseline cepat sebelum mencoba model yang lebih berat.
- **MultinomialNB + CountVectorizer** mampu mengklasifikasikan teks spam dan bukan spam sederhana dengan benar, termasuk untuk teks baru yang belum pernah dilihat saat training.